

1 Basic

1.1 Default

```

1 #include<bits/stdc++.h>
2 #define int long long int
3 #define all(x) (x).begin(), (x).end()
4 #define INF 1e15+9
5 #define DBG(x) cout<<(#x " = ") <<x<<endl;
6 #define pb push_back
7 #define fastio ios_base::sync_with_stdio(0);cin.tie(0);
8 using namespace std;
9 const int maxn = 2e5+5;
10 const int MOD = 1e9+7; // 998244353;
11 typedef pair<int,int> P;
12
13 void solve(){
14     // Do Something...
15 }
16 signed main(){
17     fastio
18     int t=1;
19     cin>>t;
20     while(t-->0) solve();
21 }

```

2 DataStructure

2.1 Mo

```

1 // Mo's with update
2 // discretization + odd/even optimize
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int maxn = 1e6+6;
6
7 int n,q,bks,sum,L,R;
8 vector<int> v(maxn), cnt(maxn);
9 vector<pair<int,int>> mod;
10 bool cmp(array<int,4> a, array<int,4> b){
11     if(a[0] / bks == b[0] / bks){
12         if(a[1] / bks == b[1] / bks){
13             return a[2] < b[2];
14         }
15     }
16     else{

```

```

16         return a[1] / bks < b[1] / bks;
17     }
18 }
19 return (a[0] / bks) < (b[0] / bks);
20 }
21 void ADD(int x){
22     if(++cnt[x] == 1) sum++;
23 }
24 void SUB(int x){
25     if(--cnt[x] == 0) sum--;
26 }
27 void MOD(int x){
28     auto [id, val] = mod[x]; // mod_id = x
29     if(id<=R && id>=L){
30         ADD(val);
31         SUB(v[id]);
32     }
33     swap(mod[x].second, v[id]);
34 }
35 int main(){
36     ios_base::sync_with_stdio(0); cin.tie(0);
37     cin>>n>>q;
38     bks = pow(n, (double)2/(double)3); // block = n^(1.5)
39     for(int i=1;i<=n;i++) cin>>v[i];
40     vector<array<int,4>> Q;
41     vector<int> ans(q,-1);
42     int tid = -1, l, r, T = -1;
43     for(int i=0;i<q;i++){
44         char c;
45         cin>>c>>l>>r;
46         if(c == 'Q'){
47             Q.push_back({l, r, tid, i});
48         }
49         else{
50             mod.push_back({l, r}); // replace (id=L) with (color=r)
51             tid++;
52         }
53     }
54     L = 1, R = 0;
55     sort(Q.begin(),Q.end(),cmp);
56     for(int i=0;i<Q.size();i++){
57         auto [l,r,ti,ii] = Q[i];
58         while(L > l) ADD(v[--L]);
59         while(R < r) ADD(v[++R]);
60         while(L < l) SUB(v[L--]);
61         while(R > r) SUB(v[R--]);
62         while(T < ti) MOD(++T);
63         while(T > ti) MOD(T--);

```

```

64     ans[ii] = sum;
65 }
66 for(int i:ans){
67     if(i!=-1) cout<<i<<"\n";
68 }
69 }

```

2.2 Lazy Seg

```

1 #include<bits/stdc++.h>
2 #define int long long int
3 using namespace std;
4 const int maxn = 2e5+5;
5
6 struct SegTree{
7     int sum[maxn*4], tag[maxn*4];
8     void init(int n){
9         for(int i=0; i<=n*4; i++){
10             sum[i] = 0;
11             tag[i] = 0;
12         }
13     }
14     int L(int x){
15         return (x<<1);
16     }
17     int R(int x){
18         return (x<<1|1);
19     }
20     void pull(int id){
21         sum[id] = sum[L(id)] + sum[R(id)];
22     }
23     void addTag(int id, int l, int r, int k){
24         tag[id] += k;
25         int len = r-l+1;
26         sum[id] += k*len;
27     }
28     void pushTag(int id, int l, int r){
29         int mid = (l+r)>>1;
30         addTag(L(id), l, mid, tag[id]);
31         addTag(R(id), mid+1, r, tag[id]);
32         tag[id] = 0;
33     }
34     void build(int id, int l, int r, vector<int> &v){
35         if(l == r){
36             sum[id] = v[l];
37             tag[id] = 0;
38             return;

```

```

39     }
40     int mid = (l+r)>>1;
41     build(L(id), l, mid, v);
42     build(R(id), mid+1, r, v);
43     pull(id);
44 }
45 void range_update(int id, int l, int r, int ll, int rr, int val){
46     if(ll <= l && r <= rr){
47         addTag(id, l, r, val);
48         return;
49     }
50     pushTag(id, l, r);
51     int mid = (l+r)>>1;
52     if(ll <= mid) range_update(L(id), l, mid, ll, rr, val);
53     if(mid < rr) range_update(R(id), mid+1, r, ll, rr, val);
54     pull(id);
55 }
56 int range_query(int id, int l, int r, int ll, int rr){
57     if(l>=ll && r<=rr) return sum[id];
58     pushTag(id, l, r);
59     int mid = (l+r)>>1, res=0;
60     if(ll<=mid) res += range_query(L(id), l, mid, ll, rr);
61     if(mid<rr) res += range_query(R(id), mid+1, r, ll, rr);
62     return res;
63 }
64 }segtree;
65 // segtree.init(n);
66 // vector<int> v(n+1); (indexed at 1!!!)
67 // segtree.build(1, 1, n, v);
68 // range update + range query

```

2.3 BIT query

```

1 // N = max value, LOGN = log(N)
2 int bit_search(int v){
3     int sum = 0, pos = 0;
4     for(int i=LOGN; i>=0; i--){
5         if(pos + (1 << i) < N and sum + bit[pos + (1 << i)] < v){
6             sum += bit[pos + (1 << i)];
7             pos += (1 << i);
8         }
9     }
10    // may use bit_query to check:
11    // if(que(pos+1) < v) return 1e9;
12    return pos + 1; // +1 because 'pos' will have position of largest
13    // value less than 'v'

```

2.4 PBDS

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 template<typename T> using rbt = tree<T, null_type, less<T>, rb_tree_tag
   , tree_order_statistics_node_update>;
5 int main(){
6     rbt<int> t; //declare
7 }
8
9 /*
10 不支援重複值 (需要的話可用左推+值來處理)
11 支援set, map之操作
12 find_by_order(k) : 像陣列一樣回傳第 k 個值。(0-based, pointer)
13 order_of_key(k) : 回傳 k 是集合裡第幾大。(0-based)
14
15 T 資料型別
16 null_type //當作 map 使用的時候要對應什麼資料型態,
17             //要當作 set 就用 null_type
18 less<T> // key value 要用什麼方式比較
19 */

```

2.5 Treap

```

1 #include <bits/stdc++.h>
2 #define int long long
3 using namespace std;
4
5 struct node{ // support range reverse, range sum query
6     node *l, *r;
7     int key, val, sum, pri, size, rev;
8     node(int k, int v) : l(0), r(0), key(k), val(v), sum(v), pri(rand())
9     , size(1), rev(0){};
10    void up();
11    void down();
12 };
13
14 node *merge(node *a, node *b){
15     if(!a || !b) return a ? a : b;
16     if(a -> pri < b -> pri){
17         a -> down();
18         a -> r = merge(a -> r, b);
19         a -> up();
20         return a;
21     }
22 }

```

```

21 else{
22     b -> down();
23     b -> l = merge(a, b -> l);
24     b -> up();
25     return b;
26 }
27 }
28
29 void split(node *o, node *&a, node *&b, int k){ // split by key
30     if(!o)
31         a = b = 0;
32     else{
33         o -> down();
34         if(o -> key < k){
35             a = o;
36             split(o -> r, a -> r, b, k);
37         }
38         else{
39             b = o;
40             split(o -> l, a, b -> l, k);
41         }
42         o -> up();
43     }
44 }
45
46
47 void insert(node *&root, int k, int v){
48     node *a, *b;
49     split(root, a, b, k);
50     root = merge(a, merge(new node(k, v), b));
51 }
52
53 bool erase(node *&o, int k){ // erase T[k]
54     if(!o)
55         return 0;
56     if(o -> key == k){
57         node *t = o;
58         o = merge(o -> l, o -> r);
59         delete t;
60         return 1;
61     }
62     node *&t = k < o -> key ? o -> l : o -> r;
63     if(erase(t, k)) return o -> up(), 1;
64     else return 0;
65 }
66
67 void node :: up(){
68     size = 1;

```

```

69     sum = val;
70     if(l) {
71         //l -> down();
72         size += 1 -> size;
73         sum += 1 -> sum;
74     }
75     if(r) {
76         //r -> down();
77         size += r -> size;
78         sum += r -> sum;
79     }
80 }
81
82 void node :: down(){
83     if(rev){
84         swap(l, r);
85         if(l) l -> rev ^= 1;
86         if(r) r -> rev ^= 1;
87         rev = 0;
88     }
89 }
90
91 inline int size(node *o){
92     return o ? o -> size : 0;
93 }
94
95 int Rank(node *& root, int val){// Number of elements smaller than val.
96     node *a, *b;
97     split(root, a, b, val);
98     int res = size(a);
99     root = merge(a, b);
100    return res;
101 }
102
103 void split2(node *o, node *&a, node *&b, int k){ // split by size
104     if(!o)
105         a = b = 0;
106     else{
107         o -> down();
108         if(k >= size(o -> l) + 1){
109             a = o;
110             int nk = k - (size(o -> l) + 1);
111             split2(o -> r, a -> r, b, nk);
112         }
113         else{
114             b = o;
115             split2(o -> l, a, b -> l, k);
116         }

```

```

117         o -> up();
118     }
119 }
120
121 node *kth(node *&root, int k){ // find T[k]
122     node *a, *b, *c;
123     split2(root, a, c, k);
124     split2(a, a, b, k - 1);
125     root = merge(a, merge(b, c));
126     return b;
127 }
128
129 void reverse(node *&root, int l, int r){
130     node *a, *b, *c;
131     split2(root, a, b, l - 1);
132     split2(b, b, c, r - l + 1);
133     b -> rev ^= 1;
134     root = merge(a, merge(b, c));
135 }
136
137 int query(node *&root, int l, int r){
138     node *a, *b, *c;
139     split2(root, a, b, l - 1);
140     split2(b, b, c, r - l + 1);
141     b -> down();
142     int res = b -> sum;
143     root = merge(a, merge(b, c));
144     return res;
145 }
146
147 void update(node *&root, int pos, int x){ // let T[pos] = x
148     erase(root, pos);
149     insert(root, pos, x);
150 }
151
152 signed main(){
153     node *T(nullptr);
154     int n, q;
155     cin >> n >> q;
156     for(int i=1;i<=n;i++){
157         int tmp;
158         cin >> tmp;
159         insert(T, i, tmp);
160     }
161     while(q--){
162         int op;
163         cin >> op;
164         if(op == 1){ // update

```

```

165     int pos, x;
166     cin >> pos >> x;
167     update(T, pos, x);
168 }
169 else{ // query
170     int l, r;
171     cin >> l >> r;
172     cout << query(T, l, r) << '\n';
173 }
174 }
175 }

```

2.6 DSU set

```

1  /*
2  This DSU supports the following operations :
3      1 x y, Union the sets containing x and y. If they are already in the
4      same set, ignore.
5      2 x y, Move x to the set containing y. If they are already in the
6      same set, ignore.
7      3 x, Return the number of elements and the sum of elements in the
8      set containing x.
9  */
10 #include <bits/stdc++.h>
11 #define int long long
12 using namespace std;
13 const int maxn = 2e5+5;
14
15 int realIndex[maxn];
16 int p[maxn];
17 int ele[maxn];
18 int sum[maxn];
19 int cnt;
20
21 void init(int n){
22     cnt = n + 1;
23     for(int i=1;i<=n;i++){
24         p[i] = i;
25         ele[i] = 1;
26         sum[i] = i;
27         realIndex[i] = i;
28     }
29 }
30
31 int find(int x){
32     if(p[x] == x)
33         return x;

```

```

31     else
32         return p[x] = find(p[x]);
33 }
34
35 void merge(int x, int y){
36     x = realIndex[x];
37     y = realIndex[y];
38     int px = find(x), py = find(y);
39     if(px == py)
40         return;
41     p[px] = py;
42     sum[py] += sum[px];
43     ele[py] += ele[px];
44 }
45
46 void moveXToY(int x, int y){
47     if(find(realIndex[x]) == find(realIndex[y]))
48         return;
49     sum[find(realIndex[x])] -= x;
50     ele[find(realIndex[x])] -= 1;
51     realIndex[x] = cnt++;
52     sum[realIndex[x]] = x;
53     ele[realIndex[x]] = 1;
54     p[realIndex[x]] = realIndex[x];
55     merge(x, y);
56 }
57
58 void output(int x){
59     x = realIndex[x];
60     x = find(x);
61     cout << ele[x] << ' ' << sum[x] << endl;
62 }
63
64 signed main(){
65     int n, q;
66     while(cin >> n >> q){
67         init(n);
68         while(q--){
69             int type;
70             cin >> type;
71             if(type == 1){
72                 int x, y;
73                 cin >> x >> y;
74                 merge(x, y);
75             }
76             else if(type == 2){
77                 int x, y;
78                 cin >> x >> y;

```

```

79         moveXToY(x, y);
80     }
81     else{
82         int x;
83         cin >> x;
84         output(x);
85     }
86 }
87 }
88 }

```

2.7 Per Seg

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 struct Per_seg{
4     int id,nn;
5     vector<int> ln,rn,val,root;
6     Per_seg(int n, vector<int> &arr) : nn(n), id(0){
7         root.resize(1);
8         ln.resize(32*n);
9         rn.resize(32*n);
10        val.resize(32*n);
11        root[0] = build(0, nn-1, arr);
12    }
13    int build(int l, int r, vector<int> &arr){
14        int rt = ++id, mid = (l+r)>>1;
15        if(l==r){
16            val[rt] = arr[l];
17            return rt;
18        }
19        ln[rt] = build(l, mid, arr);
20        rn[rt] = build(mid+1, r, arr);
21        val[rt] = val[ln[rt]] + val[rn[rt]]; // pull
22        return rt;
23    }
24    int update(int pre_id, int l, int r, int pos, int v){
25        int rt = ++id, mid = (l+r)>>1;
26        ln[rt] = ln[pre_id];
27        rn[rt] = rn[pre_id];
28        val[rt] = val[pre_id];
29        if(l==r){
30            val[rt] = v;
31            return rt;
32        }
33        if(pos<=mid) ln[rt] = update(ln[pre_id], l, mid, pos, v);
34        else rn[rt] = update(rn[pre_id], mid+1, r, pos, v);

```

```

35        val[rt] = val[ln[rt]] + val[rn[rt]]; // pull
36        return rt;
37    }
38    int query2(int o, int l, int r, int ql, int qr){
39        if(l>=ql && r<=qr) return val[o];
40        int mid = (l+r)>>1;
41        if(qr<=mid) return query2(ln[o], l, mid, ql, qr);
42        else if(ql>mid) return query2(rn[o], mid+1, r, ql, qr);
43        return query2(ln[o], l, mid, ql, qr) + query2(rn[o], mid+1, r,
44            ql, qr);
45    }
46    // ---for outer---
47    int clone(int ver){
48        int rt = ++id;
49        ln[rt] = ln[root[ver]];
50        rn[rt] = rn[root[ver]];
51        val[rt] = val[root[ver]];
52        root.push_back(rt);
53        return rt;
54    }
55    void point_update(int ver, int pos, int newVal) {
56        int newRoot = update(root[ver], 0, nn-1, pos, newVal);
57        // root.push_back(newRoot); // copy
58        root[ver] = newRoot; // replace
59    }
60    int range_query(int ver, int l, int r){
61        // root.push_back(root[ver]); // copy
62        return query2(root[ver], 0, nn-1, l, r);
63    }
64    };
65    /*
66    0-indexed!!
67    Per_seg seg(n, arr); - build
68    */

```

2.8 DSU f

```

1 struct DSU {
2     vector<int> f, siz;
3
4     DSU(){}
5     DSU(int n){
6         init(n);
7     }
8
9     void init(int n){
10        f.resize(n);

```

```

11     iota(f.begin(), f.end(), 0);
12     siz.assign(n, 1);
13 }
14
15 int find(int x){
16     while (x != f[x]) {
17         x = f[x] = f[f[x]];
18     }
19     return x;
20 }
21
22 bool same(int x, int y){
23     return find(x) == find(y);
24 }
25
26 bool merge(int x, int y){
27     x = find(x);
28     y = find(y);
29     if (x == y) {
30         return false;
31     }
32     siz[x] += siz[y];
33     f[y] = x;
34     return true;
35 }
36
37 int size(int x){
38     return siz[find(x)];
39 }
40 };
41
42 // USE: DSU dsu(n);

```

2.9 SparseTable

```

1 const int MAXN = 5e5 + 5;
2 const int lgN = 20;
3
4 struct SP{ // 0-based
5     vector<int> Sp[lgN];
6     void build(int n, vector<int>& a){
7         for(int i=0;i<n;i++)
8             Sp[0].push_back(a[i]);
9         for(int h=1;h<lgN;h++){
10             int len = (1 << (h - 1)), i = 0;
11             for(; i + len < n; i++)
12                 Sp[h].push_back(max(Sp[h-1][i], Sp[h-1][i+len]));

```

```

13         for(; i < n; i++)
14             Sp[h].push_back(Sp[h-1][i]);
15     }
16 }
17 int query(int l, int r){
18     int lg = __lg(r - l + 1);
19     int len = (1 << lg);
20     return max(Sp[lg][l], Sp[lg][r - len + 1]);
21 }
22 };

```

2.10 01trie

```

1 // for xor
2 const int N = 1e5+5;
3 int tot, trie[41*N][2], n; // need reset tot, trie
4 int find(int x){
5     int p=0, sum=0;
6     for(int i=40;i>=0;i--){
7         int id=(x>>i)&1;
8         if(trie[p][id^1]){ // here, choose id^1
9             sum=sum*2+1;
10            p=trie[p][id^1];
11        }
12        else{
13            sum=sum*2; // here, choose id
14            p=trie[p][id];
15        }
16    }
17    return sum;
18 }
19 // fixed:
20 void insert(int x){
21     int p=0;
22     for(int i=40;i>=0;i--){
23         int id=(x>>i)&1;
24         if(!trie[p][id]) trie[p][id]=++tot;
25         p=trie[p][id];
26     }
27 }

```

2.11 Dy Seg

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 struct Node {
5     unordered_map<int, int> mp; // 記錄頻率
6     int tag = 0;
7     int l, r;
8     Node* left = nullptr;
9     Node* right = nullptr;
10 };
11
12 Node* build(int l, int r) {
13     Node* node = new Node();
14     node->l = l;
15     node->r = r;
16     return node;
17 }
18
19 void pushdown(Node* node) { // push tags
20     if (node->tag) {
21         if (!node->left) node->left = build(node->l, (node->l + node->r) / 2);
22         if (!node->right) node->right = build((node->l + node->r) / 2 + 1, node->r);
23         // Update child
24         node->left->mp.clear();
25         node->right->mp.clear();
26         int mid = (node->l + node->r) / 2;
27         int left_len = mid - node->l + 1;
28         int right_len = node->r - mid;
29         node->left->mp[node->tag] = left_len;
30         node->right->mp[node->tag] = right_len;
31         node->left->tag = node->tag;
32         node->right->tag = node->tag;
33
34         node->tag = 0;
35     }
36 }
37
38 // 將 [l, r] 區間內的數字設為 val
39 void range_update(Node* node, int l, int r, int val) {
40     if (l <= node->l && node->r <= r) {
41         node->mp.clear();
42         node->mp[val] = node->r - node->l + 1;
43         node->tag = val; // tag!
44         return;
45     }
46
47     pushdown(node);
48     int mid = (node->l + node->r) / 2;
49
50     if (l <= mid) {
51         if (!node->left) node->left = build(node->l, mid);
52         range_update(node->left, l, r, val);
53     }
54     if (r > mid) {
55         if (!node->right) node->right = build(mid + 1, node->r);
56         range_update(node->right, l, r, val);
57     }
58
59     // push_up
60     node->mp.clear();
61     if (node->left) {
62         for (auto& [key, val] : node->left->mp) {
63             node->mp[key] += val;
64         }
65     }
66     if (node->right) {
67         for (auto& [key, val] : node->right->mp) {
68             node->mp[key] += val;
69         }
70     }
71 }
72
73 unordered_map<int, int> query(Node* node, int l, int r) {
74     if (l <= node->l && node->r <= r) return node->mp;
75
76     pushdown(node);
77     int mid = (node->l + node->r) / 2;
78     unordered_map<int, int> result;
79
80     if (l <= mid && node->left) {
81         auto left_result = query(node->left, l, r);
82         for (auto& [key, val] : left_result) {
83             result[key] += val;
84         }
85     }
86
87     if (r > mid && node->right) {
88         auto right_result = query(node->right, l, r);
89         for (auto& [key, val] : right_result) {
90             result[key] += val;
91         }
92     }
93
94     return result;
95 }
96

```



```

97 void recycle(Node* &node) {
98     if (node == nullptr) return;
99     if (node->left) recycle(node->left);
100    if (node->right) recycle(node->right);
101    delete node;
102    node = nullptr;
103 }
104
105 void delete_range(Node* node, int l, int r) {
106     // delete range [l,r]
107     if (l <= node->l && node->r <= r) {
108         node->mp.clear();
109         node->tag = 0;
110         recycle(node->left);
111         recycle(node->right);
112         return;
113     }
114     pushdown(node);
115     int mid = (node->l + node->r) / 2;
116     if (node->left && l <= mid) {
117         delete_range(node->left, l, r);
118     }
119     if (node->right && r > mid) {
120         delete_range(node->right, l, r);
121     }
122     // push_up
123     node->mp.clear();
124     if (node->left) {
125         for (auto& [key, val] : node->left->mp) {
126             node->mp[key] += val;
127         }
128     }
129     if (node->right) {
130         for (auto& [key, val] : node->right->mp) {
131             node->mp[key] += val;
132         }
133     }
134 }
135
136
137 int main() {
138     int n, q;
139     cin >> n >> q;
140     Node* root = build(1, n);
141
142     while (q--) {
143         int op, l, r, x;
144         cin >> op;

```

```

145         if (op == 1) {
146             cin >> l >> r >> x; // set [l,r] to x
147             range_update(root, l, r, x);
148         } else {
149             cin >> l >> r; // query [l,r] every number's frequency
150             unordered_map<int, int> freq = query(root, l, r);
151             for (auto& [key, val] : freq) {
152                 cout<<key<<" "<<val<<endl;
153             }
154             delete_range(root, l, r);
155         }
156     }
157 }

```

3 Geometric

3.1 Rectangle Union Area

```

1 const int maxn = 1e5 + 10;
2 struct rec{
3     int t, b, l, r;
4     //t頂 · b底 · l左 · r右邊界點
5 } r[maxn];
6 int n, cnt[maxn << 2];
7 long long st[maxn << 2], ans = 0;
8 vector<int> x, y;
9 vector<pair<pair<int, int>, pair<int, int>>> v;
10 void modify(int t, int l, int r, int ql, int qr, int v) {
11     if (ql <= l && r <= qr) cnt[t] += v;
12     else {
13         int m = (l + r) >> 1;
14         if (qr <= m) modify(t << 1, l, m, ql, qr, v);
15         else if (ql >= m) modify(t << 1 | 1, m, r, ql, qr, v);
16         else modify(t << 1, l, m, ql, m, v), modify(t << 1 | 1, m, r, m,
17             qr, v);
18     }
19     if (cnt[t]) st[t] = y[r] - y[l];
20     else if (r - l == 1) st[t] = 0;
21     else st[t] = st[t << 1] + st[t << 1 | 1];
22 }
23 int main() {
24     cin >> n;
25     for (int i = 0; i < n; i++) {
26         //輸入個個長方形的上下左右界
27         cin >> r[i].l >> r[i].r >> r[i].b >> r[i].t;
28         if (r[i].l > r[i].r) swap(r[i].l, r[i].r);

```

```

28     if (r[i].b > r[i].t) swap(r[i].b, r[i].t);
29     x.push_back(r[i].l);
30     x.push_back(r[i].r);
31     y.push_back(r[i].b);
32     y.push_back(r[i].t);
33 }
34 sort(x.begin(), x.end());
35 sort(y.begin(), y.end());
36 x.erase(unique(x.begin(), x.end()), x.end());
37 y.erase(unique(y.begin(), y.end()), y.end());
38 for (int i = 0; i < n; i++) {
39     r[i].l = lower_bound(x.begin(), x.end(), r[i].l) - x.begin();
40     r[i].r = lower_bound(x.begin(), x.end(), r[i].r) - x.begin();
41     r[i].b = lower_bound(y.begin(), y.end(), r[i].b) - y.begin();
42     r[i].t = lower_bound(y.begin(), y.end(), r[i].t) - y.begin();
43     v.emplace_back(make_pair(r[i].l, 1), make_pair(r[i].b, r[i].t));
44     v.emplace_back(make_pair(r[i].r, -1), make_pair(r[i].b, r[i].t));
45     ;
46 }
47 sort(v.begin(), v.end(), [](pair<pair<int, int>, pair<int, int>> a,
48     pair<pair<int, int>, pair<int, int>> b){
49     if (a.first.first != b.first.first) return a.first.first < b.
50         first.first;
51     return a.first.second > b.first.second;
52 });
53 for (int i = 0; i < v.size(); i++) {
54     if (i) ans += (x[v[i].first.first] - x[v[i - 1].first.first]) *
55         st[1];
56     modify(1, 0, y.size(), v[i].second.first, v[i].second.second, v[
57         i].first.second);
58 }
59 cout << ans << '\n';
60 return 0;
61 }

```

3.2 TheLeastCoverCircle

```

1 const double eps = 1e-10, pi = acos(-1);
2 struct Circle{
3     pdd o;
4     double r;
5 }c;
6 vector<pdd> p;
7 int R, n, r;
8 pdd operator+(pdd a, pdd b){
9     return {a.F + b.F, a.S + b.S};
10 }

```

```

11 pdd operator-(pdd a, pdd b){
12     return {a.F - b.F, a.S - b.S};
13 }
14 pdd operator*(pdd a, double b){
15     return {a.F * b, a.S * b};
16 }
17 pdd operator/(pdd a, double b){
18     return {a.F / b, a.S / b};
19 }
20 double operator*(pdd a, pdd b){
21     return a.F * b.S - a.S * b.F;
22 }
23 int judge(double a, double b){
24     if (fabs(a-b) < eps) return 0;
25     if (a < b) return -1;
26     return 1;
27 }
28 pdd rotate(pdd a, double b){
29     return {a.F*cos(b)+a.S*sin(b), -a.F*sin(b)+a.S*cos(b)};
30 }
31 double lens(pdd a, pdd b){
32     double dx = b.F - a.F, dy = b.S - a.S;
33     return sqrt(dx*dx + dy*dy);
34 }
35 pdd intersection(pdd p, pdd v, pdd q, pdd w){//求交點
36     pdd u = p - q;
37     double t = w*u/(v*w);
38     return p + v * t;
39 }
40 pair<pdd, pdd> bisector(pdd a, pdd b){//求中垂線
41     pdd p = (a + b) / 2.0;
42     pdd v = rotate(b - a, pi / 2.0);
43     return {p, v};
44 }
45 Circle circle(pdd a, pdd b, pdd c){ //三點求圓
46     auto n = bisector(a, b), m = bisector(a, c);
47     pdd o = intersection(n.F, n.S, m.F, m.S);
48     double r = lens(o, a);
49     return {o, r};
50 }
51 void solve(){
52     p.clear();
53     cin >> n;
54     p.resize(n);
55     //輸入所有的點
56     for (int i = 1; i <= n; i++){
57         cin >> p[i].F >> p[i].S;
58     }

```

```

59 random_shuffle(all(p));
60 c = {p[0], 0};
61 for(int i = 1; i <= n; i++){
62     if (judge(c.r, lens(c.o, p[i])) == -1){
63         c = {p[i], 0};
64         for (int j = 0; j < i; j++){
65             if (judge(c.r, lens(c.o, p[j])) == -1){
66                 c = {(p[i] + p[j]) / 2.0, lens(p[i], p[j]) / 2.0};
67                 for (int k = 0; k < j; k++){
68                     if (judge(c.r, lens(c.o, p[k])) == -1){
69                         c = circle(p[i], p[j], p[k]);
70                     }
71                 }
72             }
73         }
74     }
75 }
76 //c: {圓心, 半徑}
77 if (n == 1) c = {(p[0] + p[1]) / 2.0, 0};
78 cout << setprecision(9) << fixed;
79 cout << -c.o.F << " " << -c.o.S << endl;
80 }

```

3.3 Closetpair

```

1 typedef pair<ll, ll> pii;
2 #define x first
3 #define y second
4 ll dd(const pii& a, const pii& b) {
5     ll dx = a.x - b.x, dy = a.y - b.y;
6     return dx * dx + dy * dy;
7 }
8 const ll inf = 1e18;
9 //在一點對陣列的[l, r]間找最近點對
10 ll dac(vector<pii>& p, int l, int r) {
11     if (l >= r) return inf;
12     int m = (l + r) / 2;
13     ll d = min(dac(p, l, m), dac(p, m + 1, r));
14     vector<pii> t;
15     for (int i = m; i >= l && p[m].x - p[i].x < d; i--)
16         t.push_back(p[i]);
17     for (int i = m + 1; i <= r && p[i].x - p[m].x < d; i++)
18         t.push_back(p[i]);
19     sort(t.begin(), t.end(),
20         [](pii& a, pii& b) { return a.y < b.y; });
21     int n = t.size();
22     for (int i = 0; i < n - 1; i++)

```

```

23         for (int j = 1; j < 4 && i + j < n; j++)
24             // 這裡可以知道是哪兩點是最小點對
25             d = min(d, dd(t[i], t[i + j]));
26     return d;
27 }
28 // 給一堆點，求最近點對的距離「的平方」。
29 ll closest_pair(vector<pii>& pp) {
30     sort(pp.begin(), pp.end());
31     return dac(pp, 0, pp.size() - 1);
32 }

```

3.4 Gramh

```

1 //define pdd (double/int)
2 int cross(pdd a, pdd b){
3     return a.first*b.second - a.second*b.first;
4 }
5
6 pdd operator-(pdd a, pdd b){
7     return {a.first - b.first, a.second - b.second};
8 }
9
10 double operator*(pdd a, pdd b){
11     return a.first * b.second - a.second * b.first;
12 }
13
14 // ps是所有的點，要去重!!!
15 vector<pdd> convexHull(vector<pdd>& ps) {
16     sort(all(ps));
17     vector<pdd> hull;
18     if (ps.size() <= 2) {
19         return ps;
20     }
21     for (int i = 0; i < 2; i++) {
22         int s = hull.size();
23         for (pdd p : ps) {
24             while (hull.size() - s >= 2 && cross(hull.back() - hull[hull
25                 .size() - 2], p - hull[hull.size() - 2]) < 1e-10) {
26                 hull.pop_back();
27             }
28             hull.pb(p); //push_back
29         }
30         hull.pop_back();
31         reverse(all(ps));
32     }
33     return hull;

```

4 Graph

4.1 LCA tarjan

```

1 // Tarjan (offline, O(n + m))
2 void dfs(int now, int p, int dep){
3     d[now] = dep; // d[i] = depth of i, be careful about "root should
4         // set to 0/1"
5     for(int nxt:g[now]){
6         if(nxt==p) continue;
7         dfs(nxt, now, dep+1);
8         connect(now,nxt); // connect son "to" its parent
9         vis[nxt] = 1;
10    }
11    // Deal with query
12    for(auto i:q[now]){
13        int nxt = i.first; // query has {now,nxt}
14        int id = i.second; // query_id
15        if(vis[nxt]){
16            qans[id] = find_root(nxt);
17        }
18    }
19 }

```

4.2 KuhnMunkres

```

1 const int N = 1005;
2 struct KM { // 0-base, maximum weight matching
3     ll w[N][N], hl[N], hr[N], slk[N];
4     int fl[N], fr[N], pre[N], qu[N], ql, qr, n;
5     bool vl[N], vr[N];
6     void init(int _n) {
7         n = _n;
8         for (int i = 0; i < n; ++i)
9             fill_n(w[i], n, -INF);
10    }
11    void add_edge(int a, int b, ll wei) {
12        w[a][b] = wei;
13    }
14    bool Check(int x) {
15        if (vl[x] = 1, ~fl[x])
16            return vr[qu[qr++] = fl[x]] = 1;
17        while (~x) swap(x, fr[fl[x] = pre[x]]);
18        return 0;
19    }
20    void bfs(int s) {

```

```

21     fill_n(slk, n, INF), fill_n(vl, n, 0), fill_n(vr, n, 0);
22     ql = qr = 0, qu[qr++] = s, vr[s] = 1;
23     for (ll d;;) {
24         while (ql < qr)
25             for (int x = 0, y = qu[ql++]; x < n; ++x)
26                 if (!vl[x] && slk[x] >= (d = hl[x] + hr[y] - w[x][y])) {
27                     if (pre[x] = y, d) slk[x] = d;
28                     else if (!Check(x)) return;
29                 }
30         d = INF;
31         for (int x = 0; x < n; ++x)
32             if (!vl[x] && d > slk[x]) d = slk[x];
33         for (int x = 0; x < n; ++x) {
34             if (vl[x]) hl[x] += d;
35             else slk[x] -= d;
36             if (vr[x]) hr[x] -= d;
37         }
38         for (int x = 0; x < n; ++x)
39             if (!vl[x] && !slk[x] && !Check(x)) return;
40     }
41 }
42 ll solve() {
43     fill_n(fl, n, -1), fill_n(fr, n, -1), fill_n(hr, n, 0);
44     for (int i = 0; i < n; ++i)
45         hl[i] = *max_element(w[i], w[i] + n);
46     for (int i = 0; i < n; ++i) bfs(i);
47     ll res = 0;
48     for (int i = 0; i < n; ++i) res += w[i][fl[i]];
49     return res;
50 }
51 } graph;
52
53 /*
54 Usage:
55 1. graph.init(n);
56 2. graph.add_edge(a, b, wei);
57 3. ll ans = graph.solve();
58 4. for (int i = 0; i < n; ++i) {
59     cout << "Matching " << i << " to " << graph.fl[i] << " with weight "
60         << graph.w[i][graph.fl[i]] << endl;
61 }
62 if want to find minimum weight matching, just negate the weight of all
63 edges
64 and then the answer will be the negative of the maximum weight matching
65 */

```

4.3 Hungarian

```

1 // Maximum Cardinality Bipartite Matching
2 // Worst case O(nm)
3
4 struct Graph{
5     static const int MAXN = 5003;
6     vector<int> G[MAXN];
7     int n, match[MAXN], vis[MAXN];
8
9     void init(int _n){
10         n = _n;
11         for (int i=0; i<n; i++) G[i].clear();
12     }
13
14     bool dfs(int u){
15         for (int v:G[u]){
16             if (vis[v]) continue;
17             vis[v]=true;
18             if (match[v]==-1 || dfs(match[v])){
19                 match[v] = u;
20                 match[u] = v;
21                 return true;
22             }
23         }
24         return false;
25     }
26
27     int solve(){
28         int res = 0;
29         memset(match, -1, sizeof(match));
30         for (int i=0; i<n; i++){
31             if (match[i]==-1){
32                 memset(vis, 0, sizeof(vis));
33                 if ( dfs(i) ) res++;
34             }
35         }
36         return res;
37     }
38 } graph;

```

4.4 MinCostMaxFlow

```

1 const int N = 2005;
2 struct MinCostMaxFlow { // 0-base
3     struct Edge {

```

```

4         ll from, to, cap, flow, cost, rev;
5     } *past[N];
6     vector<Edge> G[N];
7     int inq[N], n, s, t;
8     ll dis[N], up[N], pot[N];
9     bool BellmanFord() {
10         fill_n(dis, n, INF), fill_n(inq, n, 0);
11         queue<int> q;
12         auto relax = [&](int u, ll d, ll cap, Edge *e) {
13             if (cap > 0 && dis[u] > d) {
14                 dis[u] = d, up[u] = cap, past[u] = e;
15                 if (!inq[u]) inq[u] = 1, q.push(u);
16             }
17         };
18         relax(s, 0, INF, 0);
19         while (!q.empty()) {
20             int u = q.front();
21             q.pop(), inq[u] = 0;
22             for (auto &e : G[u]) {
23                 ll d2 = dis[u] + e.cost + pot[u] - pot[e.to];
24                 relax(e.to, d2, min(up[u], e.cap - e.flow), &e);
25             }
26         }
27         return dis[t] != INF;
28     }
29     void solve(int _s, int _t, ll &flow, ll &cost, bool neg = true) {
30         s = _s, t = _t, flow = 0, cost = 0;
31         if (neg) BellmanFord(), copy_n(dis, n, pot);
32         for (; BellmanFord(); copy_n(dis, n, pot)) {
33             for (int i = 0; i < n; ++i) dis[i] += pot[i] - pot[s];
34             flow += up[t], cost += up[t] * dis[t];
35             for (int i = t; past[i]; i = past[i]->from) {
36                 auto &e = *past[i];
37                 e.flow += up[t], G[e.to][e.rev].flow -= up[t];
38             }
39         }
40     }
41     void init(int _n) {
42         n = _n, fill_n(pot, n, 0);
43         for (int i = 0; i < n; ++i) G[i].clear();
44     }
45     void add_edge(ll a, ll b, ll cap, ll cost) {
46         G[a].pb(Edge{a, b, cap, 0, cost, (ll)G[b].size()});
47         G[b].pb(Edge{b, a, 0, 0, -cost, (ll)G[a].size() - 1});
48     }
49 };
50
51 /*

```

```

52 Usage:
53 1. mcmf.init(n); // Initialize, n = number of nodes (0-based indexing)
54 2. mcmf.add_edge(u, v, cap, cost); // Add edge with capacity `cap` and
   per-unit cost `cost`
55 3. mcmf.solve(s, t, flow, cost, neg); // Run Min-Cost Max-Flow
56    - s = source
57    - t = sink
58    - flow = maximum flow (output)
59    - cost = corresponding minimum cost (output)
60    - neg = whether negative-cost edges may exist; default = true.
61 Example:
62     MinCostMaxFlow mcmf;
63     mcmf.init(4);
64     mcmf.add_edge(0,1,2,1);
65     mcmf.add_edge(0,2,1,2);
66     mcmf.add_edge(1,2,1,0);
67     mcmf.add_edge(1,3,1,3);
68     mcmf.add_edge(2,3,2,1);
69     ll flow, cost;
70     mcmf.solve(0, 3, flow, cost);
71     // flow = 3, cost = 8
72 */

```

4.5 Bridge

```

1 //for undirected graph, find bridge
2 const int N = 1e6+5;
3
4 vector<pair<int,int> > edge(N); //{u,v} ->remember to input
5 vector<pair<int,int> > g[N]; //{nxt,edge_id}
6 vector<int> bridge(N);
7 int dfn[N],vis[N],low[N],id;
8 void tarjan(int now,int p){
9     dfn[now]=id++;
10    vis[now]=1;
11    low[now]=dfn[now];
12    for(auto [nxt,id]:g[now]){
13        if(nxt==p) continue;
14        if(vis[nxt]){
15            low[now]=min(low[now],dfn[nxt]); //back edge!
16        }
17        else{
18            tarjan(nxt,now);
19            low[now]=min(low[now],low[nxt]);
20            if(low[nxt]>dfn[now]){
21                bridge[id]=1;
22            }
23        }
24    }
25 }

```

```

23     }
24 }
25 }
26 signed main(){
27     // construct
28     for(int i=0;i<n;i++){
29         dfn[i]=1e9;//reset
30         low[i]=1e9;
31         vis[i]=0;
32     }
33     id=0;
34     for(int i=0;i<n;i++){
35         if(!vis[i]){
36             tarjan(i,-1);
37         }
38     }
39     // use
40 }

```

4.6 LCA fd

```

1 // online  $O(n \log n + m \log n)$ 
2 const int N = 300005;
3 int d[N],f[N][20]; // f[i][j] = i's  $2^j$  father, d[i] = depth of i
4 vector<int> g[N]; // graph
5
6 void dfs(int now,int p,int dep){
7     d[now] = dep+1;
8     for(int nxt:g[now]){
9         if(nxt==p) continue;
10        f[nxt][0] = now;
11        dfs(nxt,now,dep+1);
12    }
13    return;
14 }
15
16 int lca(int x, int y){
17     if(d[x]<d[y]) swap(x,y);
18     int k = d[x]-d[y];
19     for(int i=0;i<20;i++){
20         if(k&1) x = f[x][i];
21         k>>=1;
22     } // jump to the same depth/height
23     if(x==y) return x;
24     for(int i=19; i>=0;i--){
25         if(f[x][i]!=f[y][i]){
26             x = f[x][i];

```

```

27         y = f[y][i];
28     }
29     // find the first different -> higher is LCA
30     return f[x][0];
31 }
32 void sol(){
33     int n,m;
34     cin>>n>>m;
35     for(int i=1;i<n;i++){
36         int u,v;
37         cin>>u>>v;
38         g[u].pb(v);
39         g[v].pb(u);
40     }
41     dfs(1,0,0); // arbitrarily choose a root, here choose 1 as root
42     for(int j=1;j<20;j++){
43         for(int i=1;i<=n;i++){
44             f[i][j] = f[f[i][j-1]][j-1];
45         } // get all f
46     }
47     // --- use lca(u,v) to get ---
48 }

```

4.7 Dinic

```

1 (a) Bounded Maxflow Construction:
2 1. add two node ss, tt
3 2. add_edge(ss, tt, INF)
4 3. for each edge u -> v with capacity [l, r]:
5     add_edge(u, tt, l)
6     add_edge(ss, v, l)
7     add_edge(u, v, r-l)
8 4. see (b), check if it is possible.
9 5. answer is maxflow(ss, tt) + maxflow(s, t)
10 -----
11 (b) Bounded Possible Flow:
12 1. same construction method as (a)
13 2. run maxflow(ss, tt)
14 3. for every edge connected with ss or tt:
15     rule: check if their rest flow is exactly 0
16 4. answer is possible if every edge do satisfy the rule;
17 5. otherwise, it is NOT possible.
18 -----
19 (c) Bounded Minimum Flow:
20 1. same construction method as (a)
21 2. answer is maxflow(ss, tt)
22 -----

```

```

23 (d) Bounded Minimum Cost Flow:
24 * the concept is somewhat like bounded possible flow.
25 1. same construction method as (a)
26 2. answer is maxflow(ss, tt) + ( $\sum l * cost$  for every edge)
27 -----
28 (e) Minimum Cut:
29 1. run maxflow(s, t)
30 2. run cut(s)
31 3. ss[i] = 1: node i is at the same side with s.
32 -----
33
34 const long long INF = 1LL<<60;
35 struct Dinic { //O(VVE), with minimum cut
36     static const int MAXN = 5003;
37     struct Edge{
38         int u, v;
39         long long cap, rest;
40     };
41
42     int n, m, s, t, d[MAXN], cur[MAXN];
43     vector<Edge> edges;
44     vector<int> G[MAXN];
45
46     void init(){
47         edges.clear();
48         for ( int i = 0 ; i < n ; i++ ) G[i].clear();
49         n = 0;
50     }
51
52     // min cut start
53     bool side[MAXN];
54     void cut(int u) {
55         side[u] = 1;
56         for ( int i : G[u] ) {
57             if ( !side[ edges[i].v ] && edges[i].rest ) cut(edges[i].v);
58         }
59     }
60     // min cut end
61
62     int add_node(){
63         return n++;
64     }
65
66     void add_edge(int u, int v, long long cap){
67         edges.push_back( {u, v, cap, cap} );
68         edges.push_back( {v, u, 0, 0LL} );
69         m = edges.size();
70         G[u].push_back(m-2);

```

```

71     G[v].push_back(m-1);
72 }
73
74 bool bfs(){
75     fill(d,d+n,-1);
76     queue<int> que;
77     que.push(s); d[s]=0;
78     while (!que.empty()){
79         int u = que.front(); que.pop();
80         for (int ei : G[u]){
81             Edge &e = edges[ei];
82             if (d[e.v] < 0 && e.rest > 0){
83                 d[e.v] = d[u] + 1;
84                 que.push(e.v);
85             }
86         }
87     }
88     return d[t] >= 0;
89 }
90
91 long long dfs(int u, long long a){
92     if ( u == t || a == 0 ) return a;
93     long long flow = 0, f;
94     for ( int &i=cur[u]; i < (int)G[u].size() ; i++ ) {
95         Edge &e = edges[ G[u][i] ];
96         if ( d[u] + 1 != d[e.v] ) continue;
97         f = dfs(e.v, min(a, e.rest) );
98         if ( f > 0 ) {
99             e.rest -= f;
100             edges[ G[u][i]^1 ].rest += f;
101             flow += f;
102             a -= f;
103             if ( a == 0 ) break;
104         }
105     }
106     return flow;
107 }
108
109 long long maxflow(int _s, int _t){
110     s = _s, t = _t;
111     long long flow = 0, mf;
112     while ( bfs() ){
113         fill(cur,cur+n,0);
114         while ( (mf = dfs(s, INF)) ) flow += mf;
115     }
116     return flow;
117 }
118 } dinic;

```

4.8 2sat tarjan

```

1  const int N = 2005;
2  int low[N],dfn[N],color[N],ins[N]; //要開兩倍大
3  // color[x] 是 x 所在的 scc 的topo逆序。
4  vector<int> g[N];
5  int dfsClock,sccCnt;
6  stack<int> stk;
7  void tarjan(int u) {
8      low[u] = dfn[u] = ++dfsClock;
9      stk.push(u); ins[u] = true;
10     for (const auto &v : g[u]) {
11         if (!dfn[v]) tarjan(v), low[u] = std::min(low[u], low[v]);
12         else if (ins[v]) low[u] = std::min(low[u], dfn[v]);
13     }
14     if (low[u] == dfn[u]) {
15         ++sccCnt;
16         do {
17             color[u] = sccCnt;
18             u = stk.top(); stk.pop(); ins[u] = false;
19         } while (low[u] != dfn[u]);
20     }
21 }
22
23 signed main(){
24     g[i].pb(j); // i->j
25
26     for (int i = 1; i <= (n << 1); ++i) if (!dfn[i]) tarjan(i); // run
27     tarjan, 注意0~2n-1 or 1~2n
28
29     for(int i=1;i<=n;i++){
30         if(color[i] == color[i+n]){
31             cout<<"NO"<<endl;
32             return;
33         }
34     }
35     cout<<"YES"<<endl;
36     // 找環 注意建邊方法是(i,i+1) or (i,i+n)
37
38     for(int i=1;i<=n;i++){
39         if(color[i] < color[i+n]){
40             cout<<1<<" ";
41         }
42         else cout<<0<<" ";
43     }
44     // 構造解 (注意是0~n-1還是1~n)

```


4.9 Hopcroft

```

1 // O(sqrt(V)E)
2 struct Graph{
3     static const int MAXN = 4005;
4     vector<int> G[MAXN];
5     int N1, N2, match[MAXN], dist[MAXN];
6
7     void init(int _n1, int _n2){
8         N1 = _n1;
9         N2 = _n2;
10        for (int i = 0; i < N1 + N2; i++) G[i].clear();
11    }
12
13    void addEdge(int u, int v){
14        G[u].pb(v + N1);
15    }
16
17    bool bfs(){
18        queue<int> Q;
19        for (int u = 0; u < N1; u++){
20            if (match[u] == -1){
21                dist[u] = 0;
22                Q.push(u);
23            } else {
24                dist[u] = -1;
25            }
26        }
27        bool found = false;
28        while (!Q.empty()){
29            int u = Q.front(); Q.pop();
30            for (int v : G[u]){
31                int m = match[v];
32                if (m != -1 && dist[m] != -1) continue;
33                if (m != -1){
34                    dist[m] = dist[u] + 1;
35                    Q.push(m);
36                } else {
37                    found = true;
38                }
39            }
40        }
41        return found;
42    }
43
44    bool dfs(int u){
45        for (int v : G[u]){
46            int m = match[v];

```

```

47        if (m == -1 || (dist[m] == dist[u] + 1 && dfs(m))){
48            match[u] = v;
49            match[v] = u;
50            return true;
51        }
52    }
53    dist[u] = -1;
54    return false;
55 }
56
57 int solve(){
58     int res = 0;
59     memset(match, -1, sizeof(match));
60     while (bfs()){
61         for (int u = 0; u < N1; u++){
62             if (match[u] == -1){
63                 if (dfs(u)) res++;
64             }
65         }
66     }
67     return res;
68 }
69 } graph;
70
71 /*
72 graph.init(n1, n2) : 2 parts' node numbers
73 graph.addEdge(i, j) : edge i->j (不須bias)
74 graph.solve() : return match numbers
75 graph.match[i] : match AFTER bias!
76 */

```

4.10 2e cc

```

1 // i.e. bridge tree
2 // Remember to reset vis[]
3 map<int,int> compId;
4 vector<int> g2[N];
5 void dfs(int now,int p,int iid){
6     vis[now]=1;
7     compId[now]=iid;
8     for(auto [nxt,id]:g[now]){
9         if(bridge[id]) continue;
10        if(nxt==p) continue;
11        if(!vis[nxt]) dfs(nxt,now,iid);
12    }
13 }
14 //Then, in main()

```

```

15 int iid=0;
16 for(int i=0;i<n;i++) vis[i]=0;
17 for(int i=0;i<n;i++){
18     if(!vis[i]){
19         dfs(i,-1,iid);
20         iid+=1;
21     }
22 }
23 for(int i=0;i<m;i++){
24     if(bridge[i]){
25         auto [u,v] = edge[i];
26         g2[compId[u]].pb(compId[v]);
27         g2[compId[v]].pb(compId[u]);
28     }
29 }

```

5 Math

5.1 FFT

```

1 const int N = 1e7+10;
2 const double Pi = acos(-1.0);
3 struct Complex{
4     double x,y;
5     Complex ( double xx=0, double yy=0){
6         x=xx;
7         y=yy;
8     }
9 };
10 Complex a[N], b[N];
11 Complex operator + (Complex a, Complex b) { return Complex(a.x + b.x , a
    .y + b.y);}
12 Complex operator - (Complex a, Complex b) { return Complex(a.x - b.x , a
    .y - b.y);}
13 Complex operator * (Complex a, Complex b) { return Complex(a.x * b.x - a
    .y * b.y , a.x * b.y + a.y * b.x);}
14
15 int limit=1, h=0, rev[N];
16
17 void fft(Complex *A, int flag){
18     for(int i=0; i<limit; i++){
19         if(i<rev[i]){
20             swap(A[i], A[rev[i]]);
21         }
22     }
23     for(int len=1; len<limit; len<=<=1){

```

```

24         // Len = 待合併區間的一半
25         Complex Wn(cos(Pi/len), flag*sin(Pi/len));
26         for(int R=len<<1, j=0; j<limit; j+=R){
27             Complex w(1, 0);
28             for (int k = 0; k < len; k++, w = w * Wn) {
29                 Complex x = A[j + k], y = w * A[j + len + k];
30                 A[j + k] = x + y;
31                 A[j + len + k] = x - y;
32             }
33         }
34     }
35 }
36
37 int main(){
38     int n,m; // n,m次方
39     cin>>n>>m;
40     for(int i=0; i<=n; i++) cin>>a[i].x;
41     for(int i=0; i<=m; i++) cin>>b[i].x;
42     while(limit<=n+m){
43         limit=limit<<1;
44         h++;
45     }
46     for(int i=0; i<limit; i++){
47         rev[i] = (rev[i>>1]>>1) | ((i&1)<<(h-1)); // bit reverse
48     }
49     fft(a, 1);
50     fft(b, 1); //FFT
51     for(int i=0; i<=limit; i++) a[i]=a[i]*b[i];
52     fft(a, -1); //IFFT
53     for(int i=0; i<=n+m; i++) cout<<(int)(a[i].x / limit + 0.5)<<" ";
54 }

```

5.2 RabinMiller

```

1 #include <bits/stdc++.h>
2 #define int long long
3 using namespace std;
4
5 int QuickPow(int base, int exponent, int mod){
6     if(exponent == 0)
7         return 1;
8     if(exponent == 1)
9         return base;
10    if(exponent % 2)
11        return QuickPow(base, exponent - 1, mod) * base % mod;
12    int tmp = QuickPow(base, exponent / 2, mod);
13    return tmp * tmp % mod;

```

```

14 }
15
16 bool RabinMiller(int d, int n){
17     int a = 2 + rand() % (n - 2);
18     if(QuickPow(a, n - 1, n) != 1)
19         return false;
20     int cur = QuickPow(a, d, n);
21     int nx;
22     while(d != n - 1){
23         nx = (cur * cur) % n;
24         d *= 2;
25         if(cur != 1 && cur != n - 1){
26             if(nx == 1)
27                 return false;
28         }
29         cur = nx;
30     }
31     return true;
32 }
33
34 bool isPrime(int n, int k){
35     if(n <= 1)
36         return false;
37     if(n <= 3)
38         return true;
39     if(n == 4)
40         return false;
41     if((n - 1) % 6 != 0 && (n + 1) % 6 != 0)
42         return false;
43     int d = n - 1;
44     int r = 0;
45     while(d % 2 == 0){
46         d /= 2;
47         r++;
48     }
49     for(int i=0; i<k; i++){
50         if(!RabinMiller(d, n))
51             return false;
52     }
53     return true;
54 }
55
56 signed main(){
57     int n;
58     while(cin >> n){
59         if(isPrime(n, 5)) // 預設 k = 5
60             cout << "質數" << endl;
61         else

```

```

62             cout << "非質數" << endl;
63         }
64     }

```

5.3 LinearSieve

```

1 int LeastPrimeDivisor[maxn];
2 vector<int> pr;
3
4 void LinearSieve(){
5     for(int i = 2; i < maxn; i++){
6         if(!LeastPrimeDivisor[i]) pr.push_back(i), LeastPrimeDivisor[i] = i;
7         for(int p : pr){
8             if(i * p >= maxn) break;
9             LeastPrimeDivisor[i * p] = p;
10            if(i % p == 0) break;
11        }
12    }
13 }

```

5.4 Rho

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 __int128 n;
5 __int128 gcd(__int128 a, __int128 b) {
6     if (b == 0) return a;
7     return gcd(b, a % b);
8 }
9
10 __int128 read() {
11     __int128 x = 0, f = 1;
12     char ch = getchar();
13     while (ch < '0' || ch > '9') {
14         if (ch == '-') f = -1;
15         ch = getchar();
16     }
17     while (ch >= '0' && ch <= '9') {
18         x = x * 10 + ch - '0';
19         ch = getchar();
20     }
21     return x * f;
22 }
23

```

```

24 void print(__int128 x) {
25     if (x < 0) {
26         putchar('-');
27         x = -x;
28     }
29     if (x > 9) print(x / 10);
30     putchar(x % 10 + '0');
31 }
32 bool cmp(__int128 x, __int128 y) { return x > y; }
33
34 __int128 quick_pow(__int128 x, __int128 p, __int128 mod) {
35     __int128 ans = 1;
36     while (p) {
37         if (p & 1) ans = (__int128)ans * x % mod;
38         x = (__int128)x * x % mod;
39         p >>= 1;
40     }
41     return ans;
42 }
43
44 bool Miller_Rabin(__int128 p) {
45     if (p < 2) return 0;
46     if (p == 2) return 1;
47     if (p == 3) return 1;
48     __int128 d = p - 1, r = 0;
49     while (!(d & 1)) ++r, d >>= 1;
50     for (int k = 0; k < 10; ++k) {
51         __int128 a = (__int128)rand() % (p - 2) + 2;
52         __int128 x = quick_pow(a, d, p);
53         if (x == 1 || x == p - 1) continue;
54         for (int i = 0; i < r - 1; ++i) {
55             x = (__int128)x * x % p;
56             if (x == p - 1) break;
57         }
58         if (x != p - 1) return 0;
59     }
60     return 1;
61 }
62
63 __int128 Pollard_Rho(__int128 x) {
64     __int128 s = 0, t = 0;
65     __int128 c = (__int128)rand() % (x - 1) + 1;
66     __int128 step = 0, goal = 1;
67     __int128 val = 1;
68     for (goal = 1;; goal *= 2, s = t, val = 1) {
69         for (step = 1; step <= goal; ++step) {
70             t = ((__int128)t * t + c) % x;
71             __int128 ts;

```

```

72             if (t >= s) ts = t - s;
73             else ts = s - t;
74             val = (__int128)val * ts % x;
75             if ((step % 127) == 0) {
76                 __int128 d = gcd(val, x);
77                 if (d > 1) return d;
78             }
79         }
80         __int128 d = gcd(val, x);
81         if (d > 1) return d;
82     }
83 }
84
85 set<__int128> st;
86 void fac(__int128 x) {
87     if (x < 2) return;
88     if (Miller_Rabin(x)) {
89         st.insert(x);
90         return;
91     }
92     __int128 p = x;
93     while (p >= x) p = Pollard_Rho(x);
94     while ((x % p) == 0) x /= p;
95     fac(p); fac(x);
96 }
97
98 vector<__int128> v; // prime factors
99 set<__int128> allf; // factors
100 void dfs(__int128 res, __int128 now) {
101     auto it = allf.find(now);
102     if (it != allf.end()) return;
103     allf.insert(now);
104     if (res < 2) return;
105     for (__int128 i : v) {
106         if (res % i == 0) {
107             dfs(res / i, now * i);
108         }
109     }
110 }
111
112 int main() {
113     srand((unsigned)time(NULL));
114     n = read();
115     __int128 N = (__int128)(n) * n;
116     fac(n); // find all n's prime factor
117     for (__int128 i : st) v.push_back(i);
118     dfs(N, 1);
119     // allf = all N's factors

```

```

120     for(auto i:allf){
121         print(i); cout<<endl;
122     }
123 }

```

5.5 NTT

```

1  const int N = 1e7+10;
2  const int P = 998244353, G = 3, Gi = 332748118; // primitive root = 3, Gi
   = mod inverse of 3
3
4  int fastpow(int x,int p){
5      int sum = 1;
6      while(p){
7          if(p&1) sum = sum*x%P;
8          x = x*x%P;
9          p = p>>1;
10     }
11     return sum;
12 }
13
14 int a[N], b[N], limit=1, h=0, rev[N];
15 inline void NTT(int *A, int flag) {
16     for(int i = 0; i < limit; i++)
17         if(i < rev[i]) swap(A[i], A[rev[i]]);
18     for(int len = 1; len < limit; len <= 1){
19         int Wn = fastpow( flag == 1 ? G : Gi , (P - 1) / (len <= 1));
20         for(int j = 0; j < limit; j += (len <= 1)){
21             int w = 1;
22             for(int k = 0; k < len; k++) {
23                 int x = A[j + k], y = w * A[j + k + len] % P;
24                 A[j + k] = (x + y) % P,
25                 A[j + k + len] = (x - y + P) % P;
26                 w = (w * Wn) % P;
27             }
28         }
29     }
30 }
31
32 signed main(){
33     int n,m;
34     cin>>n>>m; // n,m次方
35     for(int i=0; i<=n; i++){
36         cin>>a[i];
37         a[i]=(a[i] + P) % P;
38     }
39     for(int i=0; i<=m; i++){

```

```

40         cin>>b[i];
41         b[i]=(b[i] + P) % P;
42     }
43     while(limit<=n+m){
44         limit=limit<<1;
45         h++;
46     }
47     for(int i=0; i<limit; i++){
48         rev[i] = (rev[i>>1]>>1) | ((i&1)<<(h-1));
49     }
50     NTT(a, 1);
51     NTT(b, 1);
52     for(int i=0; i<=limit; i++) a[i]=a[i]*b[i]%P;
53     NTT(a, -1);
54
55     int inv = fastpow(limit, P - 2);
56     for(int i=0; i<=n+m; i++){
57         cout<<(a[i]*inv)%P << " ";
58     }
59 }

```

6 Misc

6.1 custom hash

```

1  #include<bits/stdc++.h>
2  #define ll long long int
3  using namespace std;
4
5  struct custom_hash {
6      static uint64_t splitmix64(uint64_t x) {
7          x += 0x9e3779b97f4a7c15;
8          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
9          x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
10         return x ^ (x >> 31);
11     }
12
13     size_t operator()(ll x) const {
14         static const uint64_t FIXED_RANDOM =
15             chrono::steady_clock::now().time_since_epoch().count();
16         return splitmix64(x + FIXED_RANDOM);
17     }
18 };
19 // unordered_map<ll, int, custom_hash> mp;

```

6.2 Rand

```

1 mt19937_64 rnd(random_device{}());
2 uniform_int_distribution<int> dist(i, j);
3 /*
4  dist(rnd) -> 取 i~j範圍內的整數
5  */

```

6.3 SG

```

1 int f[100]; // state, reset to -1
2
3 int sg(int x) {
4     if (f[x] != -1) return f[x];
5     unordered_set<int> S;
6     if (x >= 1) S.insert(sg(x - 1)); // all sub-states (suppose x can be
7         x-1, x-2, 0)
8     if (x >= 2) S.insert(sg(x - 2));
9     if (x >= 3) S.insert(sg(0));
10    for (int i = 0;; i++) {
11        if (!S.count(i)) return f[x] = i;
12    }

```

7 Strings

7.1 SuffixArray

```

1 struct SuffixArray{
2     void countSort(vector<int> &pos, vector<int> &rank){
3         int n = pos.size();
4         vector<vector<int>> cnt(n);
5         for(auto i : pos) cnt[rank[i]].push_back(i);
6         for(int i=0,idx=0;i<n;i++){
7             for(auto x : cnt[i])
8                 pos[idx++] = x;
9         }
10    }
11    void getSuffix(string s, vector<int> &pos, vector<int> &lcp){
12        s += '$';
13        int n = s.size();
14        vector<int> rank(n);
15        iota(pos.begin(), pos.end(), 0);
16        sort(pos.begin(), pos.end(), [&](int a, int b){ return s[a] < s[
            b]; });

```

```

17 // k = 0
18 for(int i=0;i<n;i++){
19     if(i == 0)
20         rank[pos[i]] = 0;
21     else
22         rank[pos[i]] = rank[pos[i-1]] + (s[pos[i]] != s[pos[i-1]]);
23 }
24 // k > 0
25 vector<int> newRank(n);
26 for(int k=0;(1<<k)<=n;k++){
27     for(int i=0;i<n;i++){
28         pos[i] = (pos[i] - (1 << k) % n + n) % n;
29         countSort(pos, rank);
30         newRank[pos[0]] = 0;
31         for(int i=1;i<n;i++){
32             pair<int,int> prev = {rank[pos[i-1]], rank[(pos[i-1]+(1<<k))%n]};
33             pair<int,int> now = {rank[pos[i]], rank[(pos[i]+(1<<k))%n]};
34             newRank[pos[i]] = newRank[pos[i-1]] + (prev!=now);
35         }
36         rank = newRank;
37     }
38     // build lcp
39     int k = 0;
40     for(int i = 0; i < n; i++){
41         int pi = rank[i];
42         int j = pos[pi-1];
43         while(i + k < n && j + k < n && s[i+k] == s[j+k]) k++;
44         lcp[pi] = k;
45         k = max(0, k-1);
46     }
47 }
48 } sa;
49
50 /*
51 vector<int> pos, lcp;
52 sa.getSuffix(s, pos, lcp);
53 */

```

7.2 KMP

```

1 vector<int> Failure(const string &s){
2     // initialize the failure function
3     vector<int> f(s.size(), 0);
4     int k = 0;
5     for (int i = 1; i < (int)s.size(); ++i) {

```

```

6         while (k > 0 && s[i] != s[k]) k = f[k - 1];
7         if (s[i] == s[k]) ++k;
8         f[i] = k;
9     }
10    return f;
11 }
12
13 vector <int> Matching(const string &s, const string &t){
14     // return 0-indexed occurrence of t in s
15     // O(N + M)
16     vector <int> f = Failure(t);
17     int l = 0; // for the index of s
18     int r = 0; // for the index of t(pattern)
19     vector <int> res ;
20     while(l < s.size()){
21         if(s[l] == t[r]){
22             l++;
23             r++;
24             if(r == t.size()){
25                 res.push_back(l - t.size());
26                 r = f[r-1];
27             }
28         }
29         else{
30             if(r == 0){
31                 l++;
32             }
33             else{
34                 r = f[r-1];
35             }
36         }
37     }
38     return res;
39 }

```

7.3 Rolling hash

```

1 const int N = 2000; // string size
2 int X = 1000000007, P = 4000000007;
3 // other primes : 8298176713 5119240589 3735751997 3218996237...
4 int s[N], p[N]; // if hash兩次 -> 開兩個s,p
5
6 void Hash(string& str){
7     s[0] = str[0];
8     p[0] = 1;
9     for(int i=1; i<str.size(); i++){
10         s[i] = (s[i-1]*X + str[i])%P;

```

```

11         p[i] = (p[i-1]*X) % P;
12     }
13     return;
14 }
15 int hash_i(int a,int b){
16     if(!a) return s[b];
17     int tmp = s[b] - s[a-1]*p[b-a+1]; // calculate H[a:b]
18     tmp=tmp%P;
19     if(tmp<0) return tmp+P;
20     return tmp;
21 }
22
23 signed main(){
24     // Hash to generate s and p
25     // hash_i to calculate s[a:b]
26     string s;
27     cin>>s;
28     Hash(s);
29     cout<<hash_i(0,2)<<" "<<hash_i(3,5);
30 }

```

ACM ICPC Team - NYCU

OverFlowers

Contents

1 Basic	1		
1.1 Default	1		
2 DataStructure	1		
2.1 Mo	1		
2.2 Lazy Seg	1		
2.3 BIT query	2		
2.4 PBDS	2		
2.5 Treap	2		
2.6 DSU set	3		
2.7 Per Seg	4		
2.8 DSU f	5		
2.9 SparseTable	5		
2.10 0ltrie	5		
2.11 Dy Seg	6		
3 Geometric	7		
3.1 Rectangle Union Area	7		
3.2 TheLeastCoverCircle	7		
3.3 Closetpair	8		
		3.4 Gramh	8
4 Graph		4 Graph	9
		4.1 LCA tarjan	9
		4.2 KuhnMunkres	9
		4.3 Hungarian	9
		4.4 MinCostMaxFlow	10
		4.5 Bridge	10
		4.6 LCA fd	11
		4.7 Dinic	11
		4.8 2sat tarjan	12
		4.9 Hopcroft	12
		4.10 2e cc	13
		5 Math	13
		5.1 FFT	13
		5.2 RabinMiller	14
		5.3 LinearSieve	14
		5.4 Rho	14
		5.5 NTT	15
		6 Misc	16
		6.1 custom hash	16
		6.2 Rand	16
		6.3 SG	16
		7 Strings	16
		7.1 SuffixArray	16
		7.2 KMP	17
		7.3 Rolling hash	17